Radiance 2 architecture

Radiance 2 technical specification, document 2

r2-spec

Version 0.2, draft of 2026-10-16

Contents

1 Introduction 2

2 Hierarchy 3

3 Memory system 8

4 Neutrino placement 13

5 Processing elements 14

6 Programming model 19

7 Open items 21

1 Introduction

This document specifies the hardware architecture of Radiance 2. It covers the hierarchy of nodes, the memory system, the placement of the Neutrinos, the processing elements and the programming model. Each block has a cost model: its capacity, its bandwidth, its transfer width, its latency and its clock ratio. The cost model is an abstraction of the hardware. It is independent of any simulator.

1.1 Scope

The configuration of Radiance 2 is open: the number of nodes, the memory sizes, the bandwidths, the latencies and the topology are not fixed. This document therefore specifies the architecture in general terms. Each configuration value is a placeholder: a parameter name, its meaning, its unit and the value "to be specified" (TBS, Section 2.4).

Values of example configurations appear only in examples. They show one possible value and are not part of the specification. The 2p family is a device in the style of Radiance 2 for the study of the pi0 model. The RTX PRO 6000 reference is a GPU that is described in the terms of this document (Section 2.5).

Document 1 specifies the Neutrino: the instructions (Section 5), the descriptor (Section 4), the tables (Section 6) and the versions (Section 7). It also specifies the task lifecycle (Section 8), the protocols (Section 9) and the rules of compute follows data (Section 3). This document does not repeat them. Document 3 tells how the simulator r2-sim models the blocks of this document in time (Section 1). Document 4 tells how a trace shows them (Section 1).

1.2 Principles

Radiance 2 must be efficient hardware on which complex kernels reach a high utilization. A complex kernel uses several kinds of PE (matrix, vector, scalar) and several memory patterns. The architecture follows five principles:

1. **One control paradigm at every level.** Every node has a Neutrino. A Neutrino runs the program of its node, starts tasks on the PEs of the node and starts programs on the child nodes. Every level uses the same instructions (Section 4).
2. **One global address space.** The high bits of an address select a subtree of the hierarchy. Each address that holds data or a signal has exactly one owner (Sections 2.2 and 3.2).
3. **Compute follows data.** A tracked task runs at the Neutrino that owns its destination range. A tracked copy into the memory of a node is therefore a pull by a DMA engine of that node (Section 5.1). An untracked task is the exception (Section 5.4).
4. **Modular composition.** Nodes, memories, buses, links and PEs are building blocks with the same interfaces at every level. A configuration puts them together (Section 2.4).
5. **Direct performance visibility.** Every memory port, bus port and link direction has its own utilization probe. A trace shows the use of each one, so that one block explains each delay.

1.3 Conventions

- The reference clock is the clock of the SMs. A cycle is an SM cycle. A block with another clock has a clock ratio (Section 2.3). A bandwidth or a rate is per cycle of the clock of its block. Latencies are in SM cycles.

- Sizes are in bytes, with IEC prefixes. A range is [*base*, *bound*).
- The word *core* names a functional unit of an SM. The Atlas is a tensor core, and the Muon is a SIMT core. The Neutrino has a scalar core. A node at the bottom of the hierarchy is an SM.
- In a tree of three levels, N_{cl} is the number of clusters of the device and N_{sm} the number of SMs of a cluster.
- In the examples, the device is `root`, a cluster is `gpc`*i* and an SM is `sm`*j*. For example, `root.gpc0.sm1` is SM 1 of cluster 0.
- The text uses the terms of the glossary (Section 4). A term in bold is defined where it appears first.

2 Hierarchy

A Radiance 2 device is a tree of nodes (Figure 1). The **device** is the root of the tree, and the **SMs** are its leaves. The nodes between them are **clusters**. The tree can have any depth: a cluster can have clusters as children. An SM holds the compute cores: the Atlas (a tensor core) and the Muon (a SIMT core). The 2p family has three levels: the device, the clusters and the SMs.

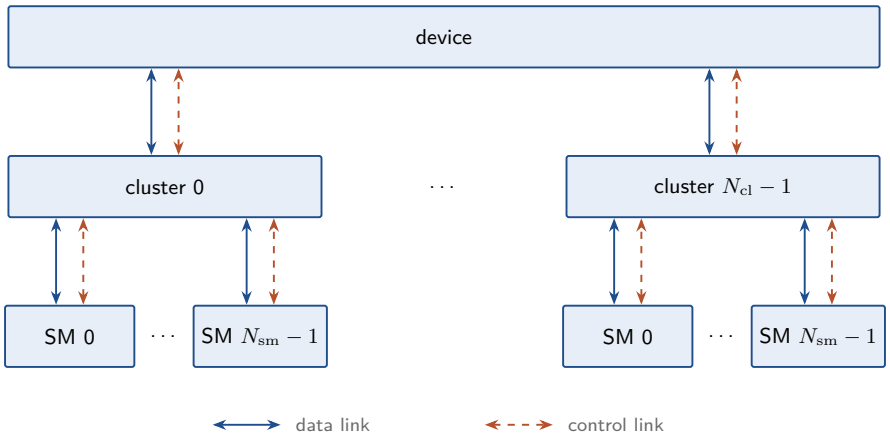


Figure 1: The hierarchy of nodes. Each child connects to the buses of its parent through one data link and one control link.

2.1 Nodes

A **node** has these elements (Figure 2):

- one Neutrino (Section 4);
- zero or more memories (Section 3.1);
- a fabric of buses and links (Section 3.4);
- zero or more PEs (Section 5);
- zero or more child nodes.

Rule R2.1 (one Neutrino). Every node must have exactly one Neutrino.

The Neutrino of a node owns and tracks the memories of the node (Section 3.2). So a node with memory always has a Neutrino.

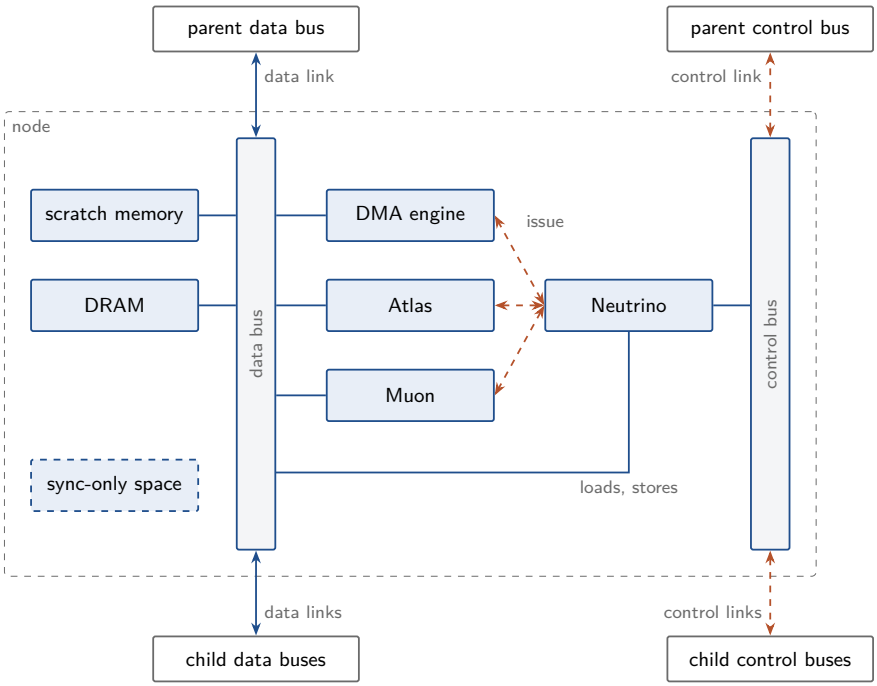


Figure 2: One node. Memories are slaves on the data bus. PEs and the Neutrino are masters. The Neutrino sends and receives messages on the control bus and starts tasks on the PEs of its node.

Each level has a role (Table 1). The roles are the usual use of the levels. The architecture lets any node have any memory and any PE.

Table 1: The levels of the hierarchy.

Level	Role	Usual elements
device	The root: the links between the clusters; signals for the whole device.	Neutrino, data bus, control bus, sync-only space. It can also hold a DRAM, a second-level memory and a DMA engine.
cluster	A group of SMs with shared memory.	Neutrino, data bus, control bus, a cluster DRAM or a cluster memory, a DMA engine, sync-only space.
SM	Compute on data in the SMEM.	Neutrino, SM data bus, local bus, SM control bus, SMEM, Atlas, tensor memory, Muon, DMA engine, sync-only space (Figure 4).

Example. The 2p family has 2 clusters of 4 SMs. Each cluster holds a cluster DRAM and a DMA engine. The RTX PRO 6000 reference has 12 clusters of 16 SMs. Its device holds the L2 and the GDDR7 memory, and its clusters hold no memory.

2.2 Address fields and own spaces

An address has 64 bits (Figure 3). The low w_{off} bits are the offset. Above the offset, the address has one **address field** per level below the device, from the top: w_1 bits for the first level, w_2 bits for the second, and so on. In a tree of three levels these are the cluster field and the SM field. The bits above the fields are 0. The sum of the widths must not be more than 64 bits.

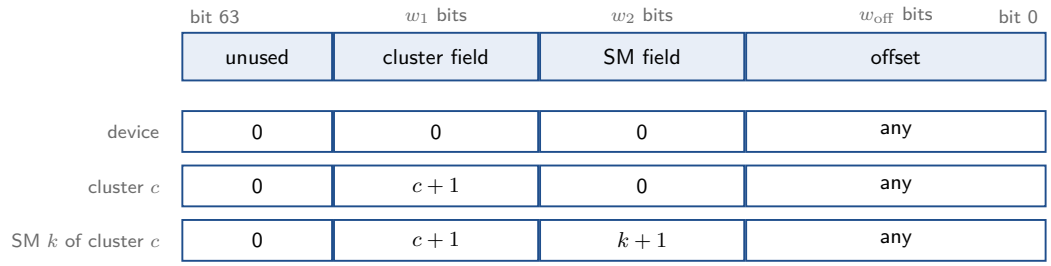


Figure 3: The fields of an address, and the field values of the own space of each kind of node.

The field values of a node are its **prefix**. The value of a child in the field of its level is its index among the children of its parent, plus 1. The fields below the level of a node are 0, so the device has 0 in every field. The **own space** of a node is the set of $2^{w_{\text{off}}}$ addresses with its prefix and 0 in the lower fields. The addresses with the prefix of a node and any values in the lower fields form the space of its subtree. A node owns ranges only inside its own space (Section 3.2).

Rule R2.2 (child count). A node must have at most $2^w - 1$ children, with w the width of the address field of its children.

Example. The 2p family has a cluster field of 2 bits, an SM field of 3 bits and an offset of 33 bits. So an address has 38 bits, and each node has 8 GiB of own space. The own space of cluster g starts at $(g+1) \times 2^{36}$. The own space of SM s of cluster g starts at $(g+1) \times 2^{36} + (s+1) \times 2^{33}$. For example, the own space of `root.gpc0.sm0` starts at `0x12_0000_0000`.

2.3 Cost model

Each block has a **cost model**: the parameters that give the time of the work on the block (Table 2). The cost model is an abstraction. It does not fix the implementation of the block.

Table 2: The parameters of the cost model of a block.

Parameter	Meaning	Unit
capacity	The bytes that a memory holds.	B
bandwidth	The bytes that a memory port, a bus port or a link direction moves per cycle of its own clock. A block with a separate write channel also has a write bandwidth.	B/cycle
rate	The operations that a compute core does per cycle of its own clock.	MAC/cycle, lane-op/cycle
transfer width	The bytes of one transfer of the block.	B
latency	The cycles from a request to its data at a memory, or the cycles that data takes through a bus or a link.	cycles
clock ratio	The clock of the block over the SM clock, as a ratio of two integers.	ratio
transaction size	The grid at which the block cuts each transfer that crosses it.	B

These parameters combine as follows:

- A block moves at most *bandwidth*/*transfer width* transfers per cycle of its own clock. A transfer of fewer bytes than the transfer width still takes one whole transfer. So a narrow master can use all the transfers of a wide block and still move few bytes.

- A block with the clock ratio c moves *bandwidth* $\times$ c bytes per SM cycle.
- A bus or a link can have no limit on its bandwidth. It then adds only its latency.
- Data that crosses several blocks moves at the bandwidth of the slowest block on its path. The latency of the path is the sum of the latencies of its blocks.

Example. An L2 bank of the RTX PRO 6000 reference reads 64 B and writes 32 B per L2 cycle, in transfers of 32 B. The L2 clock has the clock ratio 7/8. So the bank reads 56 B and writes 28 B per SM cycle. A master that moves 4 B per transfer gets only 4 B of each transfer of 32 B on such a block.

2.4 Configuration parameters

A configuration of Radiance 2 gives the tree of nodes and the parameters of every block. Table 3 lists the parameters. The value of every parameter is to be specified (TBS). Document 3 gives the format of a configuration file for r2-sim and the key of each parameter in that file (Section 10.4).

Table 3: The parameters of a configuration. Every value is TBS.

Parameter	Meaning	Unit	Value
Hierarchy and addressing (Sections 2.1 and 2.2)			
levels, children	The depth of the tree; the children of each node.	count	TBS
$w_1, \ldots, w_n$, w_{off}	The widths of the address fields, one per level below the device, and of the offset.	bits	TBS
granule	The unit of the addresses of a Neutrino: destination ranges and keys count granules (Section 2.3).	B	TBS
SM clock	The reference clock.	GHz	TBS
Memories (Section 3.1)			
memories of each node	The SMEM of an SM, the cluster memory or the cluster DRAM of a cluster, the memories of the device.	list	TBS
capacity, banks	The bytes of each memory; its number of banks.	B, count	TBS
port parameters	The read bandwidth, the write bandwidth (or one port for both), the transfer width, the latency, the clock ratio and the transaction size of each memory block.	B/cycle, B, cycles, ratio	TBS
sync-only space	The size and the place of the sync-only space of each node.	B	TBS
Fabric (Sections 3.3 and 3.4)			
buses of each node	The data buses and the control bus; for each, a crossbar or a shared bus.	list	TBS
bus parameters	The bandwidth of each port (crossbar) or of the bus, the write bandwidth, the transfer width, the latency, the clock ratio and the transaction size.	B/cycle, B, cycles, ratio	TBS
links	The data links and the control links between the buses.	list	TBS
link parameters	The bandwidth of each direction, the write bandwidth, the transfer width, the latency, the clock ratio and the transaction size.	B/cycle, B, cycles, ratio	TBS
hash links	The linear range, the physical range and the index masks of each hash link.	B, masks	TBS
Control fabric (Section 3.5)			

Parameter	Meaning	Unit	Value
message beat	The bytes of one beat of a protocol message.	B	TBS
stage depth	The messages that one stage holds.	messages	TBS
flow control	Ready/valid flow, or tickets (Section 9).	none	TBS
Neutrino (Section 4)			
tables and costs	The capacities of the tables and the costs of the instructions (Section 11).	entries, cycles	TBS
scalar core	The instructions per cycle of the scalar core; the completions per cycle of the completion ports.	1/cycle	TBS
transfer width	The transfer width of the loads and stores of the Neutrino.	B	TBS
DMA engine (Section 5.1)			
transfer width, clock ratio	The bytes that the engine moves per cycle of its clock; its clock ratio.	B, ratio	TBS
engine latency	The latency before each burst.	cycles	TBS
transfers in flight	The transfers that the engine serves at once.	count	TBS
outstanding bursts	The limit on the bursts that are issued and not done, or no limit.	count	TBS
burst size	The largest burst.	B	TBS
Atlas, the tensor core (Section 5.2)			
units	The Atlas units of an SM.	count	TBS
rates	The MAC rate for each data type.	MAC/cycle	TBS
tiles	The output tiles that the Atlas supports.	elements	TBS
tmem	The capacity of the tmem; the rate of the read port and of the write port of each of its four banks.	B, B/cycle	TBS
Muon, the SIMT core (Section 5.3)			
Muons, rate	The Muons of an SM (one per sub-partition); the BF16 lane operations per cycle of a Muon (FP32 at half).	count, lane-op/cycle	TBS
task overhead	The fixed cycles of each task before its work.	cycles	TBS
Every PE (Section 5)			
tasks in flight	The tasks that the PE runs at once.	count	TBS

2.5 Example configurations

r2-sim has two families of example configurations. Document 3 lists their files (Section 10). They show values for studies and for the comparison with a GPU. They do not fix Radiance 2.

- The 2p family: 2 clusters of 4 SMs, with a cluster DRAM in each cluster. Each SM has an SMEM of 1 MiB in 2 banks, one Atlas and one Muon. Variants change the number of SMs, add a cluster memory, enlarge the SMEM or interleave the cluster DRAM over banks.
- The RTX PRO 6000 reference: a GPU of 12 clusters of 16 SMs, with the L2 and the GDDR7 memory at the device. Its values come from the documentation of the card and from measurements on it (Section 13.3).

The examples in the other sections give their main values.

3 Memory system

The memory system of a node is its memories and its fabric. The memories hold data and signals (Section 3.1). The fabric connects the memories, the PEs and the Neutrino of the node, and the node to its parent and its children (Section 3.4).

3.1 Memories

A node can have memories. A memory has a type (Table 4), an address range and, except for a sync-only space, one or more memory blocks with ports.

Table 4: The types of memory.

Type	Owns its range	Ports	Use
scratch memory	yes	yes	Data that software places: the SMEM of an SM, a cluster memory, a memory at the device.
DRAM	yes	yes	The DRAM of the device (HBM, GDDR), at the device or as one cluster DRAM per cluster.
sync-only space	yes	no	Dependencies only: the destination ranges of joins and of spawn tasks. No memory backs it.

Memory blocks and ports. A **memory block** is a contiguous physical range with its own ports. A memory without banks is one memory block. A memory block has a **read port**. It can also have a separate **write port**: then a read never waits for a write. Without a write port, the read port serves both directions. Each port has the cost model of Section 2.3: a bandwidth, a transfer width, a latency, a clock ratio and a transaction size. A memory is a slave on one or more buses (Section 3.4). An access over any of these buses uses the same ports.

Banks. A memory with B banks is B equal contiguous memory blocks. Each bank has its own ports and its own latency. Banks alone do not interleave addresses: each bank holds one contiguous part of the range. A hash link in front of the banks spreads a range over them (Section 3.3). Without a hash link, software decides which data goes to which bank. An access to a range uses the ports of the banks that hold the range, in proportion to its bytes in each bank.

Example. In the 2p family, the SMEM of an SM is 1 MiB in two banks of 512 KiB, without a hash link. Each bank has a read port and a write port of 128 B/cycle and a latency of 10 cycles. Software puts the activations (operand A of the Atlas, the inputs of the Muon) in bank 0 and the weights (operand B) in bank 1. So each operand reads at the full bandwidth of its own bank.

Scratch memories. A scratch memory owns its range: data is in it only when a program puts it there. The **SMEM** is the scratch memory of an SM. It is a slave on the local bus and on the SM data bus (Figure 4). The Muon of the SM reads and writes it over the local bus; the Atlas reads its operands from the tmem (Section 5.2). Other nodes reach the SMEM over the SM data bus and the data link of the SM. The DMA engine of the SM copies data into it (Section 5.1). The SMEM has banks along the lanes of a warp: consecutive words go to different banks, and software can swizzle the bank of a word, so that the lanes of a warp reach different banks. These lane banks are on another dimension than the contiguous banks of a memory (Section 3.1); their number, word width and swizzle are TBS. A **cluster memory** is an optional scratch memory

on the data bus of a cluster, which the SMs of the cluster share. A scratch memory at the device is also optional, for example a second-level memory for all clusters.

Example. A variant of the 2p family adds a cluster memory of 64 MiB to each cluster. It has a read port and a write port of 1024 B/cycle and a latency of 30 cycles.

DRAM. A DRAM can be at the device, for all clusters, or at each cluster. In the second case, the DRAM of the device is non-uniform (NUMA): each cluster owns a part of it, a **cluster DRAM**. An SM reads the cluster DRAM of another cluster through the device. A tracked copy into a cluster DRAM is a task of the cluster that owns it, on a DMA engine of that cluster (Rule R5.2). An untracked copy from a DMA engine of an SM can also write into it (Section 5.4). The channels of a DRAM can be banks behind a hash link.

Example. In the 2p family, each cluster owns a cluster DRAM of 4 GiB with one port of 512 B/cycle and a latency of 200 cycles. In the RTX PRO 6000 reference, the device owns the GDDR7 memory: 64 channel banks behind a hash link. Each bank moves 2 B per DRAM cycle at the clock ratio 51/10.

Sync-only space. A **sync-only space** is an address range that a Neutrino tracks for dependencies and that no memory backs, so it has no ports. The destination ranges of joins and of spawn tasks must be in it (Rule N2.5, Section 6.2). These tasks need no PE and write no data.

Rule R3.1 (one sync-only space). A node must have at most one sync-only space. It must lie in the own space of the node.

Example. In the 2p family, every node has a sync-only space of 16 MiB at the offset `0x1_F000_0000` of its own space.

No caches. Radiance 2 has no caches. Software places all data in the scratch memories and the DRAMs (Section 3.2). Document 3 gives the cache model of r2-sim for the L2 of the RTX PRO 6000 reference (Section 8).

3.2 Address map and ownership

A node owns the ranges of its scratch memories, of its DRAM and of its sync-only space, and the linear ranges of its hash links (Section 3.3). Each range is an offset in the own space of the node (Section 2.2). The **owner** of an address is the Neutrino of the node that owns a range with the address. Ownership decides:

- the Neutrino whose tables hold a task: the owner of its destination range (compute follows data, Section 3);
- the Neutrino that resolves a dependency on a key (Section 9);
- the DMA engine of a tracked copy into a range (Rule R5.2).

Rule R3.2 (own space). Every range that a node owns must be inside the own space of the node.

Rule R3.3 (one owner). Two owned ranges must not overlap.

Rule R3.4 (owned addresses). The dependencies of a task and the destination range of a tracked task must be in owned ranges.

Address 0 is the null address: no node owns it (Rule N2.1). The fence uses the key 0 (Section 5.4).

Example. In the SoC of Radiance 2, the physical addresses start at the boot ROM at `0x10000`. So address 0 holds nothing.

There is no allocator in hardware. Software places every buffer: at build time or at run time, a program chooses the address of each range. The capacity of a memory is therefore a limit on placement. Table 5 gives an example of an address map.

Table 5: Example: a part of the address map of the 2p family. The other SMs and the other cluster follow the same pattern.

Element	Base	Size	Owner
`root.sync`	`0x01_F000_0000`	16 MiB	`root`
`root.gpc0.hbm`	`0x10_0000_0000`	4 GiB	`root.gpc0`
`root.gpc0.sync`	`0x11_F000_0000`	16 MiB	`root.gpc0`
`root.gpc0.sm0.smem.b0`	`0x12_0000_0000`	512 KiB	`root.gpc0.sm0`
`root.gpc0.sm0.smem.b1`	`0x12_0008_0000`	512 KiB	`root.gpc0.sm0`
`root.gpc0.sm0.sync`	`0x13_F000_0000`	16 MiB	`root.gpc0.sm0`
`root.gpc0.sm1.smem`	`0x14_0000_0000`	1 MiB	`root.gpc0.sm1`
`root.gpc1.hbm`	`0x20_0000_0000`	4 GiB	`root.gpc1`

3.3 Hash links

A **hash link** is a link that maps a **linear range** onto a **physical range** (Section 3.4). The node of the link owns the linear range. The banks behind the link cover the physical range. Programs and Neutrinos see only linear addresses. The hash link rewrites the address of each access that crosses it.

The rewrite has $k \geq 1$ index masks and k drop bits. For a linear address a:

- bit j of the bank index is the parity of a AND $mask_j$;
- the offset in the bank is $a - linear\ base$ without the drop bits;
- the physical address is $physical\ base + bank\ index \times bank\ size + offset$, with $bank\ size = linear\ size/2^k$.

When each mask has one bit, the drop bits are the bits of the masks. The **hash granule** is 2^b, with b the lowest bit that is set in any mask. All the addresses of one aligned hash granule go to one bank, so an access that stays in one granule uses one bank.

Rule R3.5 (bijection). The rewrite of a hash link must map the linear range one to one onto the physical range. The banks behind the link must cover the physical range exactly. The linear range must not overlap a memory block.

Rule R3.6 (no compute on a hashed range). A task of a compute core (an Atlas or a Muon) must not name an address in the linear range of a hash link. Data in a hashed range reaches a compute core through a DMA transfer.

Example. A variant of the 2p family has its cluster DRAM as 4 banks behind a hash link with the masks `0x200` and `0x400`. The bank index is address bits 9 and 10, so consecutive 512 B

granules go to banks 0, 1, 2, 3, 0, and so on. In the RTX PRO 6000 reference, the L2 is 64 banks behind a hash link. Its six masks use the address bits 8 to 20, as measured on the card.

3.4 Fabric

The **fabric** of a node is its buses and its links. The data fabric carries data: the bursts of DMA transfers, the loads and stores of the Neutrino, the operands and results of the compute cores. The control fabric carries the protocol messages between Neutrinos (Section 3.5).

Rule R3.7 (two fabrics). Every node must have at least one data bus and at least one control bus.

Rule R3.8 (separate fabrics). A data link must join two data buses. A control link must join two control buses. Data and messages never share a bus or a link.

An element connects to a bus through an **attachment**. A **master** issues requests on a bus: a PE, or the Neutrino (Section 4.1). A **slave** receives requests: a memory. A link has one attachment on each of its two buses.

Buses. A **bus** connects the attachments of one node. It is a crossbar or a shared bus:

- A crossbar has an ingress port and an egress port for each attachment, each with the bandwidth of the bus. A transfer from one attachment to another uses the ingress port of the first and the egress port of the second. Two transfers between disjoint pairs of attachments do not share a port.
- A shared bus has one bandwidth for all its transfers.

A bus has a latency. A **data bus** carries data. A **control bus** carries protocol messages. An SM has three buses (Figure 4). The SM data bus leads to the data bus of the cluster. The **local bus** connects the compute cores and the SMEM. The SM control bus carries the messages of the SM.

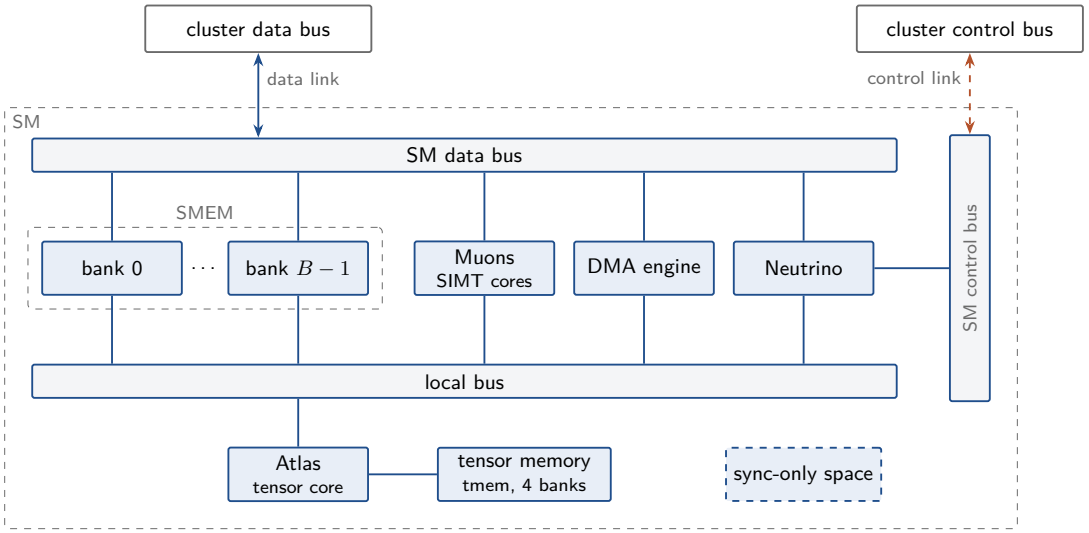


Figure 4: One SM. The SMEM banks, the Muons (one per sub-partition), the DMA engine and the Neutrino attach to the SM data bus and to the local bus. The Atlas attaches to the local bus and to the tensor memory of the SM.

Example. In the 2p family, the data bus of a cluster is a crossbar of 512 B/cycle per port. The three buses of an SM have no bandwidth limit: the SMEM ports and the data link of the SM are the limits there.

Links. A **link** joins two buses: usually a bus of its node and a bus of the parent. A link is full duplex. Each direction has a bandwidth, a transfer width, a latency and a clock ratio. A link can also join two buses of one node, for example a hash link in front of the banks of a memory.

Rule R3.9 (parent links). Every node except the device must connect to the buses of its parent through at least one data link and at least one control link.

Example. In the 2p family, the data link of an SM carries 128 B/cycle in each direction, with a latency of 10 cycles. The data link of a cluster carries 256 B/cycle in each direction, with a latency of 20 cycles.

Paths. An access goes from one attachment to another over buses and links, on the shortest path. It uses these blocks, in the order of the data:

- the read port of the source block;
- on each crossbar, the ingress port where the data enters and the egress port where it leaves;
- on each shared bus, the bus;
- on each link, the direction that the data takes;
- the write port of the destination block.

The latency of the path is the sum of the latencies of its blocks.

Example. A DMA engine of SM `sm0` of the 2p family reads the DRAM of its cluster. The data uses the port of the cluster DRAM and the ingress and egress ports of the cluster data bus. Then it uses one direction of the data link of the SM. The SM data bus has no limit. The latency of the path is 210 cycles: 200 cycles of the cluster DRAM and 10 cycles of the link. The buses add no latency.

Read and write channels. Every byte that an access moves is a read or a write, relative to the master that moves it. A read moves data from a slave to the master: the source of a DMA transfer, a load, an operand of a compute core. A write moves data from the master to a slave: the destination of a DMA transfer, a store, a result. A data bus, a data link or a memory can have two channels. The **read channel** carries the reads. The **write channel** carries the writes and has its own write bandwidth. Then a read never waits for a write. Without a write channel, one channel carries both.

Example. In the 2p family, every data bus and data link with a bandwidth limit has a write channel at its read bandwidth. Every SMEM bank has a write port. The cluster DRAMs have one port.

3.5 Control fabric

The control fabric is the control buses and the control links. It carries only the protocol messages between Neutrinos (Section 9), so a message never waits behind data. A message belongs to one of three protocol classes: invoke, dependency or hierarchy (Table 6). Each class has its own **channel**, so a message of one class never waits for a message of another class. A channel starts

with a source queue at the sender. It has one **stage** at the end of each control bus port and each control link direction. A stage holds a configured number of messages. The channel ends at the receiver. For the invoke class, the source queue is the outbound invoke table of the sender, and the receiver is the inbound invoke table of the owner (Section 6).

The channels use **ready/valid flow**. A message moves to the next stage only when that stage has room. A stage never changes the order of its messages, and a message that cannot move blocks the messages behind it. So a full receiver holds the messages back to the sender, and a full outbound invoke table stalls the program of the sender. Document 1 gives the protocols and a second flow control with tickets (Section 9).

Rule R3.10 (in order). A channel must deliver the messages from one sender to one receiver in the order in which the sender sent them.

A message has 1 to 4 beats of the message beat. An Invoke carries the descriptor of the task (Section 4). A stage stores the whole message and then sends it on.

Example. In the 2p family, the control fabric is 8 B wide: each port of a control bus and each direction of a control link moves 8 B per cycle. The message beat is 64 B, so one beat takes 8 cycles on each control hop. The control links have the latencies of the data links of the same nodes.

4 Neutrino placement

Every node has one Neutrino (Rule R2.1). Document 1 specifies the Neutrino: its instructions, its tables, its versions and its protocols. This section gives where the Neutrino sits in its node, the network that it uses, and what it does for its node.

4.1 Place in the node

The Neutrino is a master on the buses of its node (Figure 2). It uses exactly one control bus for the protocol messages, and data buses for its loads and stores.

Rule R4.1 (one control bus). The Neutrino of a node must be a master on exactly one control bus.

The **scalar core** of the Neutrino runs the program of the node. Each PE of the node has an **issue port** from the Neutrino and a **completion port** back to it. The Neutrino sends the ready tasks of a PE to it in order. A PE runs a configured number of tasks at once. The completions arrive on the completion ports, so they do not take time of the scalar core. Document 1 gives the costs of the instructions (Section 11).

Rule R4.2 (PE of the owner). The PE of a tracked task must be a PE of the node whose Neutrino owns the destination range of the task. An untracked task runs on a PE of the invoking node (Section 5.4).

A load or a store of the Neutrino goes over the data fabric to the memory block of its address. A load waits for the latency of the path to the block and for the latency of the block. Then its data takes the path back. A load is the only way for a program to see data values: control that depends on data starts with a load.

4.2 Control network

The Neutrinos talk over the control fabric (Section 3.5). A message goes from the attachment of the sender on its control bus to the attachment of the receiver, on the shortest path. A parent and a child are one control link apart. Two SMs of one cluster talk through the control bus of the cluster, and two clusters through the control bus of the device. Table 6 gives the messages by protocol class.

Table 6: The protocol classes and their messages (Section 9).

Class	Messages	Use
invoke	Invoke, VersionReset	A forwarded invoke to the owner of its destination range; the release of a version entry at another Neutrino.
dependency	Await, AwaitAck, Resolve	A dependency on a key that another Neutrino owns.
hierarchy	Start, Return	The start of a program on a child; the end of that program.

4.3 What a Neutrino does for its node

The Neutrino of a node does this work:

- It runs the program of the node. The parent starts the program with Start; the host starts the program of the device (Section 5).
- It tracks the ranges that the node owns: their producers and their versions (Sections 2 and 7).
- It accepts the invokes whose destination range the node owns, from its own program and forwarded from other Neutrinos. It forwards each invoke whose destination range another Neutrino owns (Section 3). It also holds the untracked tasks of its own program (Section 5.4).
- It starts each task on a PE of the node when the dependencies of the task are done. A join and a spawn task have no PE (Section 8).
- It registers its dependencies on the keys of other Neutrinos, and answers their registrations on its own keys (Section 9).
- It starts programs on its child nodes, and reports the end of its own program to its parent (Section 6.2).
- It loads and stores data over the data fabric.

Example. In the 2p family, the program of the device starts one program per cluster and stage. Each cluster program invokes the tasks of its SMs (Section 6.2). The invokes of a cluster program into the SMEM of an SM are forwarded invokes. The Neutrino of the SM holds these tasks and starts them on the PEs of the SM.

5 Processing elements

A **PE** is a block that runs tasks. Radiance 2 has three kinds of PE, which are peers (Table 7): the DMA engine, the Atlas (a tensor core) and the Muon (a SIMT core). Each PE has an issue port and a completion port to the Neutrino of its node (Section 4.1). Each PE is a master on one or more buses, runs a configured number of tasks at once, and has a cost model (Section 2.3).

Table 7: The processing elements.

PE	Place	Work	Cost model
DMA engine	SM, cluster, device	copies bytes from a source address to a destination address	transfer width, engine latency, transfers in flight, outstanding bursts
Atlas	SM	multiplies tiles of operands from the tmem	MAC rate per data type, tile, tmem banks
Muon	SM	runs SIMT kernels	lane operations per cycle, task overhead

5.1 DMA engine

A **DMA engine** is a PE that copies bytes from a source address to a destination address. The copy of one DMA task is a **transfer**. A node can have DMA engines. Usually each SM has one, which copies data into the SMEM and the tmem of the SM. Each cluster has one, which copies data into the cluster DRAM or the cluster memory of the cluster. A device with memory can also have one.

Example. In the 2p family, the DMA engine of an SM is a master on the local bus and on the SM data bus. The DMA engine of a cluster is a master on the data bus of the cluster.

Descriptor. A DMA task has four arguments: the source address, the destination address, the byte count and the tile, the largest burst that the program asks for. Document 1 gives the other fields of the descriptor (Section 4).

Rule R5.1 (destination range). The destination range of a DMA task must contain the bytes that the task writes.

The pull rule. **Rule R5.2 (pull).** A tracked DMA task must run on a DMA engine of the node that owns its destination range. The engine reads the source, in any node, and writes into the memory of its own node.

This rule is Rule R4.2 for DMA tasks. A copy by the engine of the destination owner is a **pull**. The rule has these consequences (Figure 5):

- A copy into the memory of another node is a forwarded invoke to the owner of the destination range. The engine of the owner pulls the data.
- The data links carry the data of a copy toward the node that receives it, as reads.
- The task lives in the tables of the owner from its invoke to its completion. There are no issue or completion messages between nodes.

An untracked DMA task is the exception: it runs on an engine of the invoking node and can write into the memory of another node (Section 5.4).

Bursts. An engine moves a transfer in **bursts**. A burst is the smallest of these sizes:

- the tile of the descriptor, or the burst size of the engine when the engine has one;
- the bytes that are left;
- the bytes to the end of the source block, and to the end of the destination block;

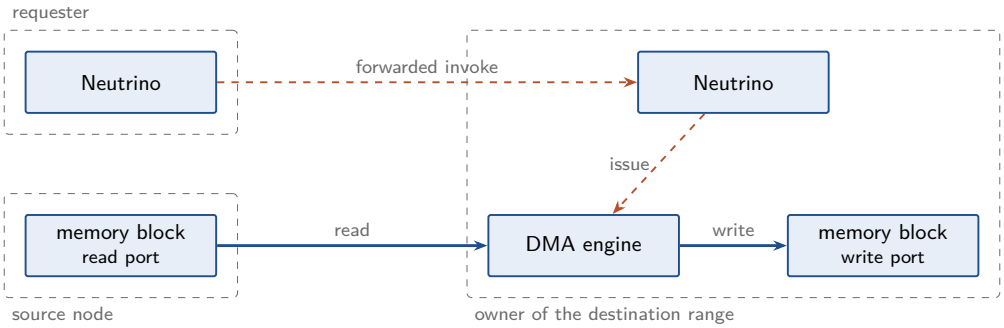


Figure 5: A copy into the memory of another node. The requester forwards the invoke to the owner of the destination range. The DMA engine of the owner reads the source and writes the destination.

- the bytes to the next multiple of each transaction size on the path (Section 2.3);
- on a path through a hash link, the bytes to the end of the hash granule (Section 3.3).

So each burst stays in one memory block, one bank and one transaction.

Flow of a transfer. An engine issues the bursts of a transfer back to back. It issues at most one transfer width per cycle of its clock. Each burst starts at the earliest one engine latency after its issue. The bursts of different transfers that share a block take turns on it. An engine serves a configured number of transfers at once. An engine can also have a limit on the bursts that are issued and not done (its credits). An engine with such a limit keeps a path busy only when

$$(\mathit{outstanding} - 1) \times \mathit{burst} \geq L \times R.$$

Here *outstanding* is the limit, L is the engine latency plus the latencies of the path, and R is the lowest bandwidth on the path.

Latency of a transfer. A transfer pays the latency of the block that holds its first source byte once, before its first burst. Its bursts are issued back to back, so it pays the engine latency and the latencies of the path about once. A write pays the latency of the fabric on its path and the time of its bytes on its blocks. The destination block adds no latency.

Example. In the 2p family, the DMA engine of an SM has an engine latency of 100 cycles. It serves 4 transfers at once and moves 256 B per cycle. A pull of 16 KiB from the cluster DRAM into the SMEM takes about $200 + 100 + 10 + 16384/128 = 438$ cycles. The terms are the latency of the DRAM, the engine latency, the latency of the SM data link and the time of the data at 128 B/cycle.

5.2 Atlas, the tensor core

The **Atlas** is the tensor core of an SM. It multiplies tiles of operands. The Neutrino coordinates it: the Atlas has no scalar sequencer of its own. An SM can have more than one Atlas unit, and each unit is a PE.

An Atlas task computes $C = A \times B$, or $C = C + A \times B$ when it accumulates. The matrices are row major, and C starts at the base of the destination range. Table 8 gives the arguments. The data types are bf16 (2 B per element), f32 (4 B), fp8 (E4M3, 1 B) and mxfp4. The type mxfp4 is the OCP microscaling FP4: E2M1 elements of 0.5 B, and one E8M0 scale of 1 B per block of 32 elements.

Table 8: The arguments of an Atlas task.

Argument	Meaning
A, B	The addresses of the two operands.
leading dimensions	The leading dimensions of A, B and C, in elements.
m, n, k	A is $m \times k$, B is $k \times n$, C is $m \times n$.
types	The data types of A, B and C.
accumulate	Add the product to C.
keep	Keep C in the tensor memory for the next task on the same destination range (Section 5.2).
tile	The output tile, rows × columns.

Tiles and rates. An Atlas computes a task one output tile at a time, over the full reduction dimension k. So the accumulation over k stays inside one task. Above the Atlas, an accumulation is a task that reads the partial sum (a dependency) and writes the new value (its destination range). The Atlas has one MAC rate per data type. A tile runs at the rate of the slower of its two operand types. An output tile takes the longer of its compute time (its MACs at this rate) and the time of its operand bytes on the banks of the tmem (Section 5.2).

Example. In the 2p family, each SM has one Atlas. Its rates are 8192 (bf16), 16384 (fp8) and 32768 (mxfp4) MAC/cycle, and the programs use tiles of 128 × 128. Each operand comes from its own bank of the tmem at 128 B/cycle. An operand of s bytes per element then feeds at most $128 \times 128/s$ MAC/cycle: 8192 for bf16 and 16384 for fp8, the full rates. An mxfp4 operand with its block scales has 0.53125 B per element, so a pure mxfp4 tile runs at most 30840 MAC/cycle: about 6 % below its rate. A tile with bf16 activations and mxfp4 weights runs at 8192 MAC/cycle. In the RTX PRO 6000 reference, each SM has four Atlas units, one per sub-partition, with 128, 256 and 512 MAC/cycle each. They read their operands from the SMEM, as the tensor cores of the card do.

Tensor memory. An SM has a dedicated **tensor memory**, the **tmem**, separate from the SMEM. The Atlas reads its operands from the tmem, and the latency of these reads is fixed. The SMEM cannot give a fixed latency: the Muon and the DMA engine also use it, and their accesses can delay an operand. The Atlas has no private matrix register file.

The tmem has four banks. Each bank has one read port and one write port (1R1W). One bank at its full read rate feeds one operand of the Atlas at the full rate of the Atlas, and its write port takes new data in the same cycles. So a bank can feed one operand while a DMA engine fills it with the next one. A DMA engine moves data into the tmem, from the SMEM or from any other memory that the engine reaches.

The Atlas accumulates an output tile itself over the reduction dimension of one task. Across the tasks of a chain (the keep flag), the partial output tile lives in the tmem: a task writes its partial tile into the tmem, and the next task on the same destination range reads it from there.

Rule R5.3 (Atlas memory). An Atlas must read its operands A and B from the tmem of its SM.

Example. In the 2p family, the tmem has 256 KiB per SM: four banks of 64 KiB, each with a read port and a write port of 128 B per cycle. A partial tile of 128 × 128 fp32 elements, read and written once per task, needs $32768/k$ B/cycle on each port at the bf16 rate, for a task over k of the reduction dimension. So at $k = 256$ the partial tile uses the read port and the write

port of one bank at their full rate.

5.3 Muon, the SIMT core

The **Muon** is the SIMT core of an SM. An SM has one Muon per sub-partition. A Muon is one SIMT backend: a warp has 16 threads, and a Muon holds up to 8 warps. One fused floating-point pipe runs the BF16 and the FP32 operations: BF16 at the full rate of 16 lane operations per cycle, FP32 at half that rate. The two types do not run at the same time.

The work of a Muon is counted in lane operations. A Muon task has these arguments (Section 4.2):

- the lane operations of the task;
- the bytes that it reads, and their address;
- the bytes that it writes at the base of the destination range.

The cost model of a Muon task has these parts:

- the lane operations at the rate of the Muon;
- the bytes in, on the path from the block of the source and on the read port of that block;
- the bytes out, on the path to the block of the destination range and on the write port of that block;
- a fixed task overhead (the launch and the reduction tail), plus the longer of the two path latencies, before the work.

The Muon reaches its SMEM over the local bus. It reaches a range of another node over the SM data bus and the data link of the SM, except the linear range of a hash link (Rule R3.6). The instruction set of the Muon is outside this document.

Example. In the 2p family, an SM has four Muons. Each Muon runs 16 BF16 or 8 FP32 lane operations per cycle, so the SM runs 64 BF16 lane operations per cycle. A task has an overhead of 50 cycles. In the RTX PRO 6000 reference, the Muon of an SM has 128 lanes and a task overhead of 20 cycles.

5.4 Untracked tasks

An invoke can mark a task as **untracked**. A task on any PE can be untracked. Document 1 gives the flag, the rules and the fence (Section 3.1). An untracked task differs from a **tracked task**, a task without the flag, in these points:

- It runs at the invoking node, on a PE of that node. So it can write any memory that the PE reaches, also the memory of another node. The pull rule does not apply to it.
- It gets no version, and no lookup finds it. So no task can depend on it, and no wait can name it (Rule N3.10).
- Its own dependencies work as for any task: it issues when they are done.
- Only a **fence** and the return of the program order later work after it. A fence is a wait on the key 0: it waits until every untracked task of the program is done (Rule N3.11). The return of a program waits for the untracked tasks of the program in the same way (Rule N8.1).

Rule R5.4 (PE of an untracked task). An untracked task must run on a PE of the invoking node. It can write any memory that this PE reaches.

The owner of the destination range does not see an untracked task. So the program orders the accesses to that range. For example, a task that reads the range comes after a fence.

Example. An SM stores its output tiles into the memory of its cluster with untracked DMA tasks on its own engine. The Neutrino of the cluster does not take part in these stores. The return of the SM program waits until the last of these stores is done, so the spawn task of its parent completes after the stores.

Note. r2-sim has untracked tasks. Its `kernels` program still uses store records for the stores of GPU kernels; document 3 tells why (Section 13.3.2).

6 Programming model

A Neutrino runs a program: a sequence of instructions to its Neutrino. The programs of the nodes form a hierarchy: the program of a node can start programs on the child nodes. This section gives the form of a program and the hierarchy of programs. It also gives the assembly text, an experimental feature.

6.1 Programs as data structures

A program is a set of data structures in memory: the sequence of its instructions and the descriptors of its tasks. Software builds them before or during the run. These data structures are the program. An **instruction** is one request of the program to its Neutrino. The Neutrino has these instructions (Section 5):

- An invoke submits a task with its **descriptor**: the opcode, the PE, the destination range, the dependencies, the label and the arguments of the task (Section 4). The arguments are those of a DMA task, an Atlas task or a Muon task (Section 5), or of a join. A tracked task gets a version from the owner of its destination range. An untracked task gets no version (Section 5.4).
- A spawn starts a program on a direct child (Section 6.2).
- A wait blocks the program until the producer of a key is done. A wait on the key 0, a fence, blocks the program until every untracked task of the program is done (Section 5.4).
- The version instructions set a version counter, clear the version table, or release one version entry at its owner (Section 5.5).
- A load or a store reads or writes bytes over the data fabric (Section 4.1).

Between these requests, the scalar core of the Neutrino does the scalar work of the program. The scalar core is an RV64 core with the Neutrino instructions (Section 1.1). Control that depends on data is scalar code over the bytes that a load returns.

Software also places every buffer at an address (Section 3.2) and cuts the work into tasks. The dependencies of a task name keys: the base of a destination range and a version (Section 7). The tool that makes these choices is outside the architecture. Document 3 gives the tools of r2-sim (Section 12.3).

6.2 Hierarchy of programs

The programs form a tree that follows the hierarchy of nodes. The host starts the *device program*. The device program starts a *cluster program* on each cluster. A cluster program can start an *SM program* on each of its SMs, or it can invoke the tasks of its SMs itself, as forwarded invokes. A **driver** is a program that invokes the tasks of a subtree.

A program starts a program on a child with a spawn. The spawn is a task in the tables of the

parent. Its destination range is in the sync-only space of the parent, and it can have dependencies. When the task is ready, the parent sends Start with the program and its epoch to the child. When the program of the child returns and its untracked tasks are done, the child sends Return, and the spawn task completes. So a wait on the destination range of the spawn waits for the child program (Section 5.3). A Start clears the version table of the child and sets its epoch (Section 7.3).

Rule R6.1 (spawn). A program must start a program only on a direct child of its node, and only when that child runs no program. The destination range of a spawn task must be in the sync-only space of the parent (Rule N2.5).

Example. In the 2p family, the program of the pi0 forward pass has two levels of drivers. For each of its three stages, the device program starts a driver on each cluster and waits for them (Listing 1). The SMs run no program: the invokes of a cluster driver into the SMEM of its SMs are forwarded invokes. Other programs have three levels. Each cluster program starts a program on each of its SMs. Each SM program invokes the tasks of its own SM, for example DMA tasks that pull data into its SMEM.

6.3 Assembly text

The **assembly text** is an experimental feature. It is a text form of the instructions that a program made in a run. It has one **listing** per program instance, with one instruction per line, in program order. A task shows as its descriptor. Software prints the text from the data structures of the program, for reading and review. It is a byproduct: no tool reads it back into a program. Document 1 gives the syntax (Section 5.8). Document 3 tells how r2-sim writes it (Section 11.6).

Listing 1 shows parts of the programs of the pi0 forward pass on the 2p family. The device program starts one driver per cluster and waits for both. The driver of cluster `gpc0` invokes a Muon task on SM `sm0`. It invokes a pull of the result into SM `sm1`, with a dependency on the version that the Muon task writes. It invokes a pull of weights from the cluster DRAM into SM `sm1`. Then it invokes a matmul that depends on both pulls, and a join on two matmul outputs. It invokes a pull from an SM of the other cluster: the dependency on the key in `root.gpc1.sm3` is a registration at that SM. Then it waits for the output of the router and loads it: the next tasks depend on these 51 bytes.

Listing 1: Example: parts of the programs of the pi0 forward pass on the 2p family, as assembly text. The parts at `...` are left out, and so are the labels. The comment `t=` gives the cycle of each instruction.

```
.program root forward    ; started by the host at cycle 0
    spawn child=root.gpc0 done=root.sync+0x0:0x200 deps=[]    ; t=0
    spawn child=root.gpc1 done=root.sync+0x200:0x200 deps=[]    ; t=27
    wait root.sync+0x0    ; t=54
    wait root.sync+0x200    ; t=9868842
    ...
.program root.gpc0 action    ; parent root, started at cycle 45831232
    vreset root.gpc0.sm1.smem+0x0    ; t=45831232
    ...
    vec dst=root.gpc0.sm0.smem+0x19800:0x19800@v1 pe=root.gpc0.sm0.muon deps=[] ops=208896 in
    ↪ =104448 out=104448    ; t=45831253
    dma dst=root.gpc0.sm1.smem+0x0:0x19800@v1 pe=root.gpc0.sm1.dma deps=[root.gpc0.sm0.smem+0
    ↪ x19800@v1] from=root.gpc0.sm0.smem+0x19800 to=root.gpc0.sm1.smem+0x0 bytes=104448 tile
    ↪ =65536    ; t=45831275
    dma dst=root.gpc0.sm1.smem+0x80000:0x11000@v1 pe=root.gpc0.sm1.dma deps=[] from=root.gpc0.
    ↪ hbm+0x0 to=root.gpc0.sm1.smem+0x80000 bytes=69632 tile=65536    ; t=45831302
    ...
```

```
mm dst=root.gpc0.sm1.smem+0x19800:0x3400@v1 pe=root.gpc0.sm1.atlas deps=[root.gpc0.sm1.smem
↪ +0x0@v1,root.gpc0.sm1.smem+0x80000@v1] a=root.gpc0.sm1.smem+0x0 b=root.gpc0.sm1.smem+0
↪ x80000 lda=1024 ldb=128 ldc=128 m=51 n=128 k=1024 types=bf16,mxfp4,bf16 acc=0 keep=0
↪ tile=128x128    ; t=45831348
...
join dst=root.gpc0.sm1.sync+0x0:0x200@v1 deps=[root.gpc0.sm1.smem+0x19800@v1,root.gpc0.sm1.
↪ smem+0x1cc00@v1]    ; t=45831414
...
dma dst=root.gpc0.sm0.smem+0x66000:0x19800@v4 pe=root.gpc0.sm0.dma deps=[root.gpc1.sm3.smem
↪ +0x53e00@v6,root.gpc0.sm0.smem+0x0@v13,root.gpc0.sm0.smem+0x66000@v3] from=root.gpc1.sm3
↪ .smem+0x53e00 to=root.gpc0.sm0.smem+0x66000 bytes=104448 tile=65536    ; t=45843901
...
wait root.gpc0.sm0.smem+0x7f800@v1    ; t=45844094
load root.gpc0.sm0.smem+0x7f800 len=51    ; t=45940675
...
```

7 Open items

These parts of the architecture are not decided yet:

1. **Configuration values.** Every value of Table 3 is TBS. A bandwidth study will set the bandwidths down the hierarchy toward the DRAM, the Atlas rate of each data type and the width of the control links. A latency study will set the latency of every block. A topology study will set the shape of the hierarchy.
2. **SMEM banks.** The SMEM has banks along the lanes of a warp, with a swizzle that software selects. The number of these banks, their word width and the swizzle are TBS (Section 3.1).
3. **Muon.** A cost model with separate counts of BF16 and FP32 lane operations in a Muon task is open (Section 5.3).
4. **Address fields.** The widths of the address fields and the constraints of the memory system on interleaving are open (Section 2.2).
5. **Encoding of the instructions.** The binary encoding of the Neutrino instructions in the code of the scalar core is not defined (Section 6.1).